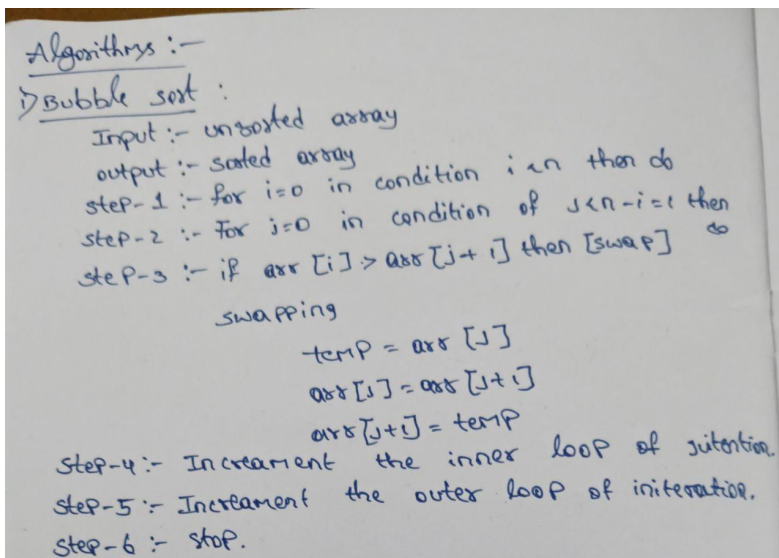


PRACTICAL-01

AIM: To implementation and time analysis of sorting algorithms. Bubble sort, Selection Sort, Insertion sort, Merge sort and Quick sort.

1. BUBBLE SORT

ALGORITHM:



PROGRAM:

```
#include <iostream>

using namespace std;

void bubbleSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}
```

```
}

int main() {
int n;

    cout << "Enter number of elements: ";
    cin >> n;

    int arr[n];
    cout << "Enter array elements: ";
    for (int i = 0; i < n; i++) {

        cin >> arr[i];
    }

    bubbleSort(arr, n);

    cout << "Sorted elements: ";
    for (int i = 0; i < n; i++) {

        cout << arr[i] << " ";
    }
    cout << endl;

    return 0;
}
```

OUTPUT:

```
Enter number of elements: 5
Enter array elements: 7
2
9
1
5
Sorted elements: 1 2 5 7 9
```

TIME COMPLEXITY:

Time complexity :-

Bubble sort :-

```

for (int i = 0; i < n-1; i++) → outer loop
{
    for (int j = 0; j < n-1-i; j++) → inner loop
    {
        if (arr[j] > arr[j+1]) {
            int temp = arr[j];
            arr[j] = arr[j+1];
            arr[j+1] = temp;
        }
    }
}

```

outer loop :- Runs i = 0 to n-1
so it executes n-1 times

inner loop :- Pass (i) iterations

0	n-1
1	n-2
⋮	⋮
n-2	1

$$(n-1) + (n-2) + \dots + 2 + 1 = 1 + 2 + \dots + (n-1)$$

$$= \frac{n(n-1)}{2} = n^2 - \frac{n}{2}$$

$$T(n) = n-1 + \frac{n^2 - n}{2} + 1 + 1 + 1 + 1$$

$$= n + \frac{n^2 - n}{2} + 3$$

$$= n + n^2 - n$$

ignore all constants

$T(n) = O(n^2) \rightarrow$ Worst case

$T(n) = O(n^2) \rightarrow$ Avg case

$T(n) = O(n) \rightarrow$ Best case

by asymptotic notation

Best case :-
Condition :- Array is in descending order.
Process :- Every comparison leads to swap.
Comparisons :- $(n-1) + (n-2) + \dots + 1 = \frac{n(n-1)}{2}$

$$\frac{n(n-1)}{2} = \frac{n^2 - n}{2} = n^2 - n$$
Time complexity = $O(n^2)$

Average case :-
Condition :- Array ele are in random order.
Process :- About half of the comparisons leads to swaps.
Comparisons :- still $\frac{n(n-1)}{2}$; Time complexity = $O(n^2)$

Worst case :-
Condition :- Array is already sorted.
Process :- Comparisons occurs but no swaps.
Time complexity = $O(n^2)$

2.SELECTION SORT

ALGORITHM:

1) selection sort :-
Input :- unsorted array
output :- sorted array
step-1 :- for $i=0$ in the condition $i < n$ then do
step-2 :- $min = i$ and inner for loop initialize
 $j = i+1$ in
step-3 :- in the condition $j = n$ then do
if $arr[j] > arr[min]$ then
 $min = j$
step-4 :- increment the inner loop $j++$
step-5 :- if $min \neq i$ then
swap ($arr[i], arr[min]$)
step-6 :- increment the outer loop $i++$
step-7 :- stop.

PROGRAM:

```
#include <iostream>
```

```
using namespace std;
```

```
void selectionSort(int arr[], int n) {
```

```
    for (int i = 0; i < n - 1; i++) {
```

```
        int minIndex = i;
```

```
        for (int j = i + 1; j < n; j++) {
```

```
            if (arr[j] < arr[minIndex]) {
```

```
                minIndex = j;
```

```
            }
```

```
        }
```

```
        int temp = arr[minIndex];
```

```
        arr[minIndex] = arr[i];
```

```
        arr[i] = temp;
```

```
    }
```

```
}
```

```
int main() {
```

S.Vigneswara Reddy(92460118133)

```
int n;

cout << "Enter number of elements: ";

cin >> n;

int arr[n];

cout << "Enter " << n << " elements: ";

for (int i = 0; i < n; i++) {
    cin >> arr[i];
}

cout << "Original array: ";

for (int i = 0; i < n; i++)
{
    cout << arr[i] << " ";
    cout << endl;
}

selectionSort(arr, n);

cout << "Sorted array: ";

for (int i = 0; i < n; i++)
{
    cout << arr[i] << " ";
    //cout << endl;
}

return 0;
}
```


OUTPUT:

```
Enter number of elements: 5
Enter 5 elements: 9
2
4
6
0
Original array: 9
2
4
6
0
Sorted array: 0 2 4 6 9
```

TIME COMPLEXITY:

1) selection sort :-

```
for (int i = 0; i < n-1; i++) {
    int min_index = i;
    for (int j = i+1; j < n; j++) {
        if (arr[j] < arr[min_index]) {
            min_index = j;
        }
    }
    int temp = arr[min_index];
    arr[min_index] = arr[i];
    arr[i] = temp;
}
```

outer loop :- The outer loop runs from $i=0$ to $n-1$ so it executes $n-1$ times.

inner loop :-

Pass	Comparisons
1	$n-1$
2	$n-2$
3	$n-3$
$\vdots$	$\vdots$
$n-1$	1

$T(n) = (n-1) + (n-2) + \dots + 1 = \frac{n(n-1)}{2}$

$T(n) = n-1 + 1 + \frac{n(n-1)}{2} + 1 + 1 + 1 + 1$

$= n + \frac{n(n-1)}{2} + 4 = 2n + \frac{n^2-n}{2} + 8$ ($\because$ by asymptotic notation and case)

$= \frac{n^2}{2} + n + 8$

$T(n) = O(n^2)$ $\rightarrow$ highest power $\rightarrow$ worst case

$T(n) = O(n^2) \rightarrow$ Best case

$T(n) = O(n^2) \rightarrow$ Avg case

2) selection sort :-

worst case :-

- $\Rightarrow$ Array is in descending order.
- $\Rightarrow$ For each position, the algo searches the entire unsorted part to find the min.
- $\Rightarrow$ comparisons $= (n-1) + (n-2) + \dots + 1 = \frac{n(n-1)}{2}$
- $\Rightarrow$ swaps $= n-1$
- $\Rightarrow$ Time complexity $= O(n^2)$

Average case :-

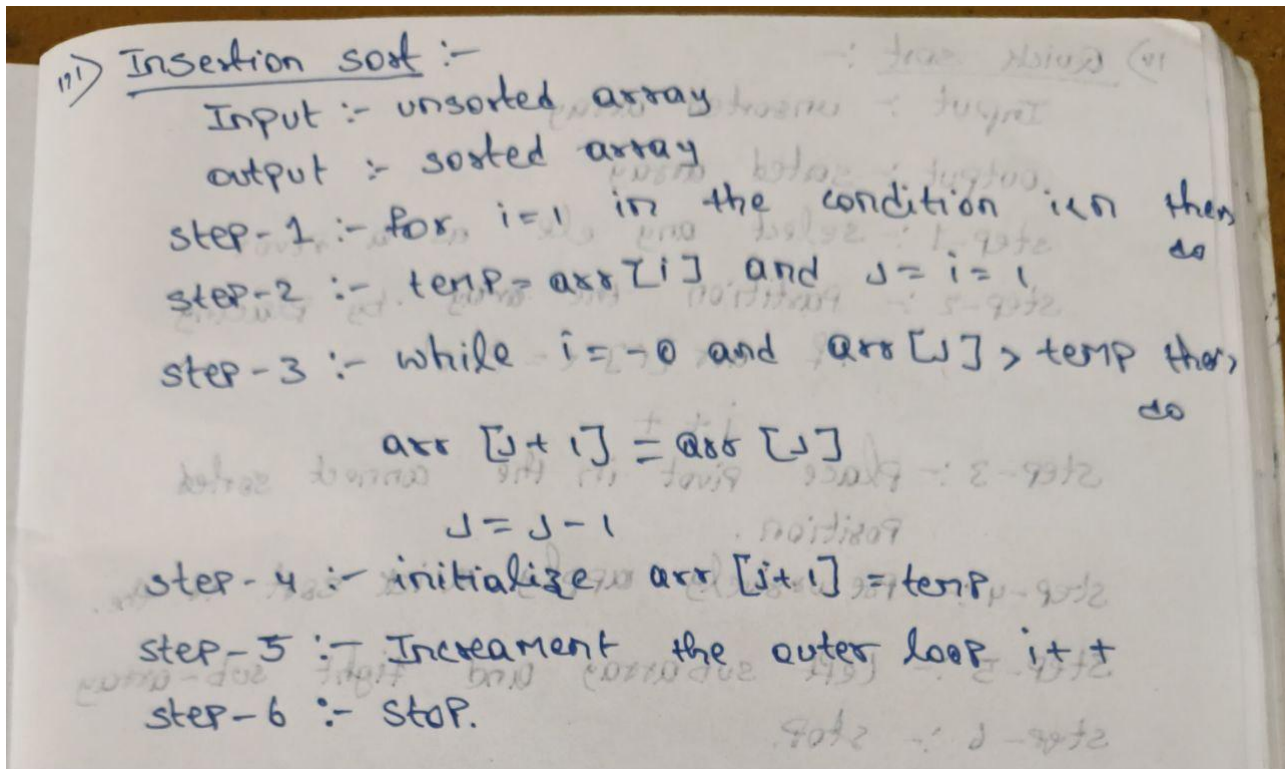
- $\Rightarrow$ Array ele are in random order.
- $\Rightarrow$ Still scans the unsorted part fully for each position.
- $\Rightarrow$ comparisons $=$ same as worst case, $\frac{n(n-1)}{2}$
- $\Rightarrow$ swaps $= n-1$
- $\Rightarrow$ Time complexity $= O(n^2)$

Best case :-

- $\Rightarrow$ Array is already sorted.
- $\Rightarrow$ Even though no swaps are needed, the algorithms still scans the unsorted part.
- $\Rightarrow$ comparisons $=$ same as worst case, $\frac{n(n-1)}{2}$
- $\Rightarrow$ swaps $= 0$
- $\Rightarrow$ Time complexity $= O(n^2)$

3.INSERTION SORT

ALGORITHM:



PROGRAM:

```
#include <iostream>
```

```
using namespace std;
```

```
void insertionSort(int arr[], int n) {
```

```
    for (int i = 1; i < n; i++) {
```

```
        int key = arr[i];
```

```
        int j = i - 1;
```

```
        while (j >= 0 && arr[j] > key) {
```

```
            arr[j+1] = arr[j];
```

```
            j--;
```

```
        }
```

```
        arr[j+1] = key;
```

```
    }
```

```
S.Vigneswara Reddy(92460118133)
```

```
}
```

```
int main() {  
    int n;  
    cout << "Enter number of elements: ";  
    cin >> n;  
  
    int arr[n];  
    cout << "Enter " << n << " elements: ";  
    for (int i = 0; i < n; i++) {  
        cin >> arr[i];  
    }  
  
    cout << "Original array: ";  
    for (int i = 0; i < n; i++)  
    {  
        cout << arr[i] << " ";  
        cout << endl;  
    }  
    insertionSort(arr, n);  
  
    cout << "Sorted array: ";  
    for (int i = 0; i < n; i++)  
    {  
        cout << arr[i] << " ";  
        //cout << endl;  
    }  
    return 0;  
}
```


OUTPUT:

```
Enter number of elements: 5
Enter 5 elements: 7
3
8
2
0
Original array: 7
3
8
2
0
Sorted array: 0 2 3 7 8
```

TIME COMPLEXITY:

insertion sort

```

2) for (int i = 1; i < n; i++) {
    int key = arr[i];
    int j = i - 1;
    while (j >= 0 & arr[j] > key) {
        arr[j+1] = arr[j];
        j--;
    }
    arr[j+1] = key;
}

```

1. Best case :- Already sorted array
 [0, 20, 30, 40, 50] → outer loop runs $n-1$ times
 → while checks only once for each element.
 $T(n) = O(n)$

2. Worst case :- Reverse sorted array
 [50, 40, 30, 20, 10]
 each ele must be compared with all previous ele and shifted.

Pass	comparisons
1	1
2	2
3	3
...	...
$n-1$	$n-1$

$$1+2+3+\dots+(n-1)$$

$$T(n) = \frac{n(n-1)}{2} = \frac{n^2-n}{2}$$

$$T(n) = O(n^2)$$

3. Avg case :- Randomly arranged array
 [30, 10, 50, 20, 40]
 an avg each ele is compared with about half of the previously sorted ele.

$$T(n) = \frac{1}{2} [1+2+3+\dots+(n-1)]$$

$$T(n) = \frac{n(n-1)}{4}$$

$$T(n) = O(n^2)$$

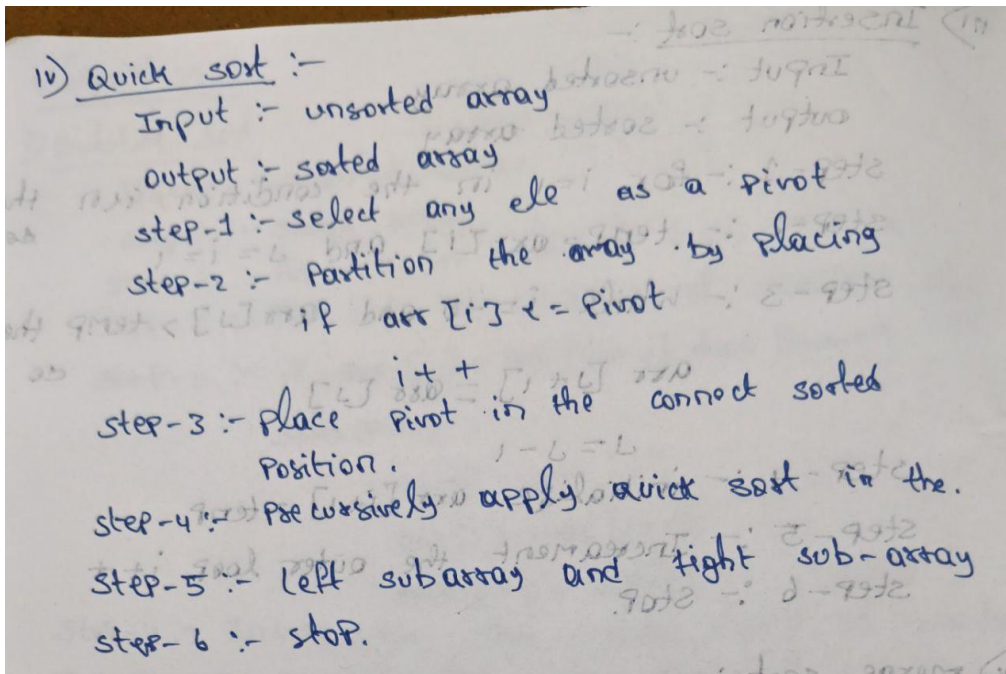
2) Insertion sort :-
worst case :-
 ⇒ Array is in descending order
 ⇒ Each new ele must be compared with all previously sorted ele and shifted.
 ⇒ comparisons & shift = $(n-1) + (n-2) + \dots + 1 = \frac{n(n-1)}{2}$
 ⇒ Time complexity = $O(n^2)$

Average case :-
 ⇒ Array ele are in random order.
 ⇒ an average each element is compared with half of the sorted portion.
 ⇒ comparisons = About $\frac{n(n-1)}{4}$
 ⇒ Time complexity = $O(n^2)$

Best case :-
 ⇒ Array is already sorted.
 ⇒ Each ele is compared only once if (no shifting needed).
 ⇒ Comparison = $n-1$
 ⇒ Time complexity = $O(n)$

4. QUICK SORT

ALGORITHM:



PROGRAM:

```
#include <iostream>
using namespace std;

int partition(int arr[], int lb, int ub) { int
    pivot = arr[lb];
    int start = lb; int
    end = ub;

    while (start < end) {
        while (arr[start] <= pivot ) {
            start++;
        }
        while (arr[end] > pivot) {
            end--;
        }
    }
}
```

```
}

if (start < end) {
    swap(arr[start], arr[end]);
}
}

swap(arr[lb], arr[end]);
return end;
}

void quickSort(int arr[], int lb, int ub) { if
(lb < ub) {
    int loc = partition(arr, lb, ub);
    quickSort(arr, lb, loc - 1); quickSort(arr,
    loc + 1, ub);
}
}

int main() {
    int n;
    cout << "Enter size of array: "; cin
    >> n;

    int arr[n];
    cout << "Enter " << n << " elements: "; for
    (int i = 0; i < n; i++) {
        cin >> arr[i];
    }
}
```

```
quickSort(arr, 0, n - 1);  
  
cout << "Sorted array: "; for  
    (int i = 0; i < n; i++) { cout  
        << arr[i] << " ";  
    }  
    return 0;  
}
```

OUTPUT:

```
Enter size of array: 5  
Enter 5 elements: 7  
3  
8  
2  
0  
Sorted array: 0 2 3 7 8
```

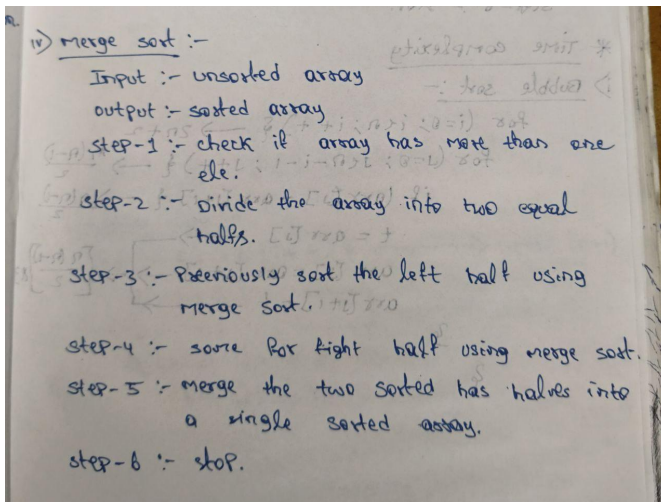
TIME COMPLEXITY:

5) Quick Sort :-
 ↳ The Pivot divides the array into two equal halves
 → $2 + \left(\frac{n}{2}\right)$
 → For sorting we use an ele at last
 $O(n)$
 $T(n) = 2 + \left(\frac{n}{2}\right) + n \rightarrow (1)$
 $n \rightarrow \frac{n}{2}$ Put in (1)
 $T\left(\frac{n}{2}\right) = 2 + \left(\frac{n}{4}\right) + \frac{n}{2} \rightarrow (2)$
 - sub (2) in (1)
 $T(n) = 2 + \left(2 + \left(\frac{n}{4}\right) + \frac{n}{2}\right) + n$
 $= 2 + \left(\frac{n}{4}\right) + \frac{3n}{2} + n$
 $T(n) = 2 + \left(\frac{n}{4}\right) + 2n \rightarrow (3)$
 $n \rightarrow \frac{n}{4}$ Put in (3)
 $T\left(\frac{n}{4}\right) = 2 + \left(\frac{n}{8}\right) + 2\left(\frac{n}{4}\right) \rightarrow (4)$
 sub (4) in (3)
 $T(n) = 2 + \left[2 + \left(\frac{n}{8}\right) + \frac{2n}{4}\right] + 2n$
 $= 2 + \left(\frac{n}{8}\right) + \frac{3n}{2} + 2n$
 $= 2 + \left(\frac{n}{8}\right) + 4n$
 $T(n) = 2 + \left(\frac{n}{8}\right) + 4n$
 $\frac{n}{2^k} = 1, n = 2^k$
 apply log on b.s
 $\log n = \log 2^k$
 $k = \log n$
 $T(n) = 2^{\log n} + \left(\frac{n}{2^{\log n}}\right) + \log(n)$
 $= n \log n + n$

$T(n) = O(n \log n) \rightarrow$ Best case
 $T(n) = O(n \log n) \rightarrow$ Avg case
 $T(n) = O(n^2) \rightarrow$ Worst case

5.MERGE SORT

ALGORITHM:



PROGRAM:

```
#include <iostream>

using namespace std;

void merge(int arr[], int left, int mid, int right) { int
    n1 = mid - left + 1;
    int n2 = right - mid;
    int L[n1], R[n2];
    for (int i = 0; i < n1; i++) L[i] = arr[left + i];
    for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) arr[k++] = L[i++]; else
            arr[k++] = R[j++];
    }
    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];
}
```



```
void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}

int main() { int
    n;
    cout << "Enter number of elements: "; cin
    >> n;
    int arr[n];
    cout << "Enter elements: ";
    for (int i = 0; i < n; i++) cin >> arr[i];

    mergeSort(arr, 0, n - 1);

    cout << "Sorted array: ";
    for (int i = 0; i < n; i++) cout << arr[i] << " ";
    return 0;
}
```

OUTPUT:

```
Enter number of elements: 5
Enter elements: 7
9
0
2
5
Sorted array: 0 2 5 7 9
```

TIME COMPLEXITY:

4) merge sort :-
The array is divided into two halves repeatedly.
no. of divisions : $\log_2 n (2 + \frac{n}{2})$
merge's each level merges all n ele
($O(n)$)
Total time $T(n) = 2 + (\frac{n}{2}) + n$
 $T(n) = 2 + (\frac{n}{2}) + n \rightarrow (1)$
 $n \rightarrow \frac{n}{2}$ Put in (1)
 $T(\frac{n}{2}) = 2 + (\frac{n}{4}) + \frac{n}{2}$
 $T(\frac{n}{2}) = 2 + (\frac{n}{4}) + \frac{n}{2} \rightarrow (2)$
Sum (1) in (1)
 $T(n) = 2(2 + (\frac{n}{4}) + \frac{n}{2}) + n$
 $= 2^2 + (\frac{n}{2}) + 2n \rightarrow (3)$
 $n \rightarrow \frac{n}{4}$ Put in (3)
 $T(\frac{n}{4}) = 2^2 + (\frac{n}{8}) + 2(\frac{n}{4}) \rightarrow (4)$
Sub (4) in (3)
 $T(n) = 2^2 [2^2 + (\frac{n}{8}) + 2(\frac{n}{4})] + 2n$
 $= 2^4 + (\frac{n}{2}) + 4n$
 $T(n) = 2^2 + (\frac{n}{2}) + 2n$

$\frac{n}{2^k} = 1$
 $n = 2^k$
Apply log both sides
 $\log n = \log 2^k$
 $T(n) = 2^{\log n} + (\frac{n}{2^{\log n}}) + \log n$
 $= n \log_2 n + \frac{n}{2^{\log_2 n}} + \log_2(n)$
 $= n \log_2 n + n$
 $T(n) = O(n \log n)$
Best case

$T(n) = O(n \log n) \rightarrow$ Avg case
 $T(n) = O(n \log n) \rightarrow$ worst case

CONCLUSION:

Bubble Sort and Selection Sort are simple but inefficient with $O(n^2)$ in all major cases. Insertion Sort improves to $O(n)$ in the best case when the array is already sorted. Quick Sort is very efficient on average with $O(n \log n)$, though it can degrade to $O(n^2)$. Merge Sort is consistently $O(n \log n)$ in all cases but requires extra space.